

LAB Semana 5: interacción, modificación y experiencia estructural

Cómo funciona el laboratorio, qué mostrar en cada criterio y qué responder, para el **LT2** solo y para el **conjunto** (cuerpo antiguo + LT2). Métodos Computacionales en Obras Civiles · UAndes · Grupo 7.

La idea en tres frases

OpenSees resuelve el edificio **una vez** por cada caso base (G, Q, EX, EY) y Python guarda todo en JSON. Unity **no calcula la estructura**: dibuja esos resultados y, como el modelo es lineal, puede **escalarlos y sumarlos al instante** cuando se mueven los sliders.

Cuando el usuario cambia algo que modifica la rigidez **K** (una barra, una sección, un apoyo), lo guardado es de otra estructura: Unity manda el modelo editado al servidor de Python y OpenSees lo **reanaliza**.

Todo lo que se muestra está **comprobado contra Python**, en los dos edificios, con tolerancias que salen del redondeo, no elegidas a ojo.

Los dos edificios

	LT2 (cuerpo nuevo)	Cuerpo antiguo	Conjunto = los dos
Planos	2024_22	2017_67	calzados, con junta de 5 cm libre
Nodos / elementos	232 / 378	326 / 559	558 / 937
IDs en el conjunto	2xxxxx (col. 69 → 200069)	1xxxxx (100537...)	LT2 = 200000 + ID
Columnas / muros con fierro	40 / 29	82 / 56	122 / 85 = 207
Hormigón	f _c 35 MPa	f _c 28 MPa	cada sección trae el suyo
Peso G	34 148.98 kN	52 600.50 kN	86 749.48 kN = la suma exacta
Columna / muro demo	5 / 9	—	200005 / 100537

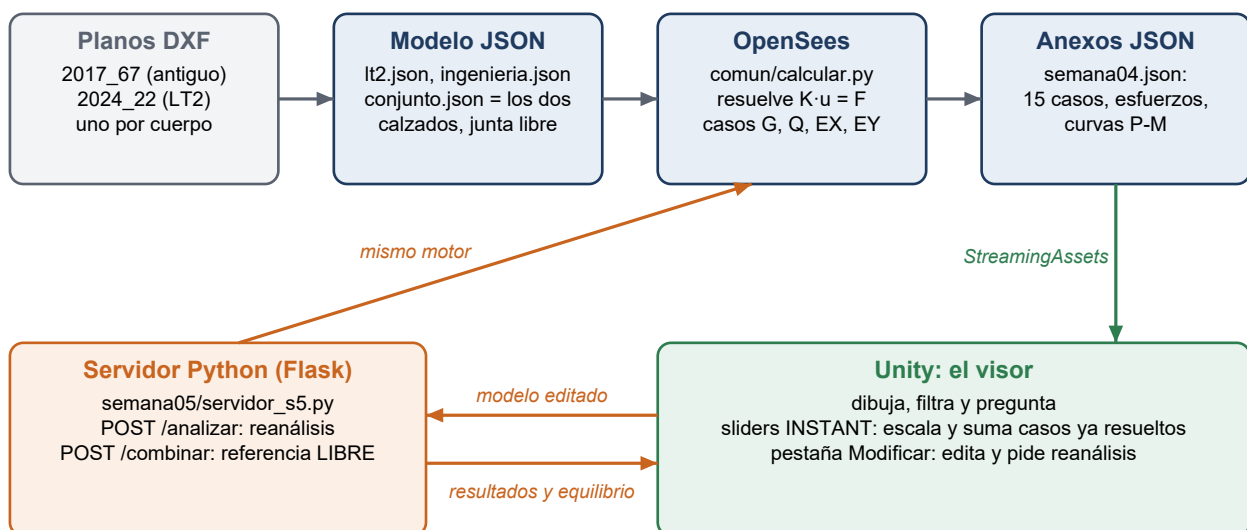
Código: repositorio [bitscochits/A1P3_Grupo_7](#), rama [semana05-lab](#) (PR #8). El LAB está en [semana05_lab/](#) y se apoya en el avance de la misma semana ([semana05/](#)). El mismo C# y el mismo Python sirven para los dos edificios: solo cambia qué anexo se carga.

La r brica, y d nde se gana cada punto

Criterio	Pt s	Qu� se muestra	C�mo sabemos que est� bien (LT2 y conjunto)
Interactividad	2	Navegar, seleccionar, capas, cambiar combinaci�n, respuesta combinada, curva P-M y mapa D/C.	<code>comparar_unity.py</code> (LT2): 488 filas contra Python, 0 fallas. Las capturas del conjunto: <code>log.errores = 0</code> .
Modificaci�n	2	Cuatro de las seis (piden dos): elemento (M1), intensidad (M2), secci�n (M3) y apoyo (M4).	<code>reanalisis_demo.py</code> y <code>comparar_anexos.py</code> en los dos edificios: TODO OK, el equilibrio cierra.
Superposici�n	2	Sliders G, Q, EX, EY que actualizan deformada, esfuerzos y punto P-M en 1 a 5 ms, sin servidor.	<code>verificar_instantanea.py --registro</code> : lo que la app escribi� coincide con Python, en el LT2 y en el conjunto.
D/C din�mica	2	El punto (P, M) recorre la curva; cambian Mn, $u = M/Mn$, PASA/NO PASA y el mapa del edificio.	El mismo verificador compara P, M, Mn, el extremo que manda y los conteos (69 barras en el LT2, 207 en el conjunto).
Criterios de rean�lisis	2	Tabla derivada de $K \cdot u = F$, cada fila comprobada en el motor.	Cs $\times 2$ da EX $\times 2$ exacto en los dos edificios.

1. C mo funciona el sistema

Es una cadena de etapas. Cada una deja un archivo JSON que la siguiente lee, as  que se puede revisar cualquier paso por separado. Los dos edificios pasan por la misma cadena.



Arriba, lo que corre en Python una vez. Abajo, lo que pasa durante la demo: Unity muestra y, si el modelo cambia, le pide al servidor que vuelva a resolver.

La regla de oro

Nunca va c lculo estructural en C#. Python arma el modelo y OpenSees lo resuelve. El JSON es la fuente de verdad y Unity lo muestra. Lo  nico que Unity hace con n meros es aritm tica sobre resultados que ya calcul  OpenSees (escalar, sumar, interpolar), y esa aritm tica est  verificada contra Python.

C mo se arma el conjunto

- Cada cuerpo se modela por separado desde sus planos (`edificios/lt2/` y `edificios/ingenieria/`) y deja su `data/modelo/.json`.

- `edificios/conjunto/armar.py` los une: aplica el **calce** (el LT2 se corre para quedar a 5 cm), **renumera** (1xxxxx el antiguo, 2xxxxx el LT2) y deja la **junta libre**: ningún nodo ni barra cruza de un cuerpo al otro. Hay 10 diafragmas, 5 por cuerpo.
- Cada sección lleva su propio hormigón (E, G, f_{pc_MPa}): el LT2 es G35 y el antiguo G28. Sin eso el LT2 correría 10.6 % más blando y el equilibrio cerraría igual, sin avisar.
- **Control:** G del conjunto = 86 749.48 kN = 52 600.50 + 34 148.98, diferencia 0.0000 kN.

Las piezas, archivo por archivo

Etapa	Archivo	Qué hace
Modelo	<code>edificios//armar.py</code>	Lee los planos y escribe <code>data/modelo/.json</code> : nodos, barras, secciones con su fierro, diafragmas y cargas.
Solver	<code>comun/servidor_opensees.py</code>	Arma K en OpenSees (<code>elasticBeamColumn</code> , diafragma rígido con <code>Transformation</code>) y aplica F.
Cálculo	<code>comun/calcular.py</code>	Resuelve G, Q, EX y EY y comprueba el equilibrio.
Capacidad	<code>comun/capacidad.py</code>	Curva P-M con una sección de fibras; corre la sección y su espejo y se queda con el menor Mn.
Anexo	<code>semana04/exportar_unity.py</code>	Los 4 casos base y las 11 combinaciones de NCh3171 (15 casos), esfuerzos por estación y curvas P-M en <code>semana04.json</code> .
Superposición	<code>semana05/superposicion.py</code>	Combina con λ libres; precalcula E1–E3 y atiende <code>POST /combinar</code> .
Servidor	<code>semana05/servidor_s5.py</code>	<code>/analizar</code> (reanálisis) y <code>/combinar</code> (referencia LIBRE), para cualquiera de los dos edificios.
Visor	<code>unity/Assets/Scripts/</code>	<code>VisorQA</code> (panel), <code>VisorSemana04.*</code> (diagramas, P-M, mapa D/C), <code>VisorSemana05.Instantanea.cs</code> (sliders INSTANT), <code>EditorEstructura</code> (Modificar).

Los cuatro casos base

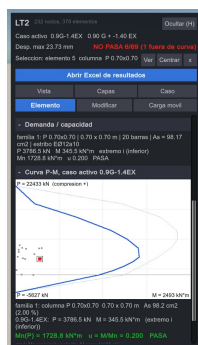
- **G:** peso propio más sobrecarga muerta (la losa baja a cada viga por su área tributaria).
- **Q:** sobrecarga de uso, 3.0 kN/m² (NCh1537, salas de clases).
- **EX y EY:** sismo estático, $F_i = C_s \cdot W_i \cdot patrón(z_i)$, con $W_i = G_i + 0.5 \cdot Q_i$ y $C_s = 0.10$. Corte basal $V = C_s \cdot W$: **3 792.28 kN** en el LT2 y **9 700.43 kN** en el conjunto.

Cualquier combinación de diseño es una suma de estos cuatro multiplicados por sus factores λ . Eso es la superposición, y funciona porque el modelo es **lineal elástico**.

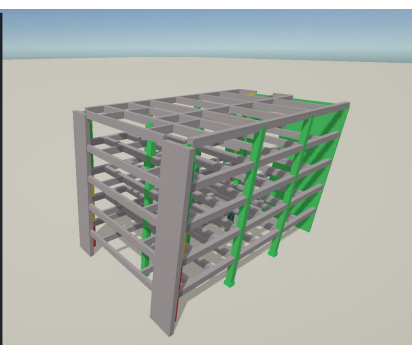
2. Interactividad (2 pts)

El enunciado pide seis cosas. Todas están en el panel de la izquierda, en las pestañas **Vista**, **Capas**, **Caso**, **Elemento**, **Modificar**, **Carga móvil**, iguales para los dos edificios.

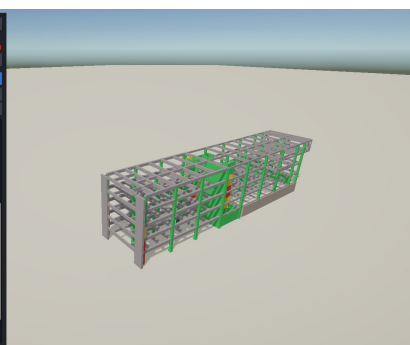
Lo que piden	Cómo se hace en la app
Navegar	Arrastrar para orbitar, rueda para acercar, F encuadra todo, C centra la selección. En Vista , el filtro por piso deja ver el interior.
Seleccionar elementos	Clic en una barra o nodo: la pestaña Elemento muestra ID, nodos, sección, material, ejes locales y esfuerzos. O <i>Ir a ID</i> (en el conjunto, con el ID largo: 2000069).
Activar/desactivar capas	Pestaña Capas : apoyos por tipo, ejes locales, áreas tributarias, diafragmas.
Cambiar combinación	Caso → <i>Casos del anexo (Semana 4)</i> : los 4 base y las 11 combinaciones.
Observar respuesta combinada	Con la combinación puesta, prender la deformada (ahora curva , con las funciones de forma del elemento) y los diagramas N, V, M.
Inspeccionar D/C	Columna demo → curva P-M con su punto. En Caso , <i>Mapa demanda/capacidad</i> pinta el edificio según $u = M/M_n$.



LT2, columna 5 en 0.9G-1.4EX: $P = 3\,786\text{ kN}$, $M = 346\text{ kN}\cdot\text{m}$, $M_n = 1\,729\text{ kN}\cdot\text{m}$, $u = 0.200$. Cabecera: NO PASA 6/69.



Conjunto, la misma columna (200005): $P = 3\,785\text{ kN}$, $M = 354\text{ kN}\cdot\text{m}$, $M_n = 1\,729\text{ kN}\cdot\text{m}$, $u = 0.205$. Cabecera: NO PASA 16/207.



Qué decir

“Todo lo que se ve viene de un JSON que calculó Python. El visor no resuelve nada: dibuja, filtra y pregunta. Y es el mismo visor para los dos edificios: cambia el anexo, no el código.”

Cómo se verifica

Unity registra lo que muestra en cada captura (*CapturaSemana05* → *registro.txt*). En el LT2, *semana05/comparar_unity.py* lo compara contra Python: **488 filas, 0 fallas** (superposición 144, M1 146, carga móvil 97, UX 58, cabeceras 43). Las 23 capturas del conjunto salieron con *log.errores* = 0.

3. Superposición en Unity (2 pts)

El enunciado pide que los sliders de los casos base actualicen **instantáneamente** la deformada, los resultados y el punto P-M.

Por qué se puede sumar

El modelo es lineal elástico: $K \cdot u = F$. Si K no cambia, la respuesta a una suma de cargas es la suma de las respuestas:

$$K \cdot (\lambda G \cdot u_G + \lambda Q \cdot u_Q + \lambda EX \cdot u_{EX} + \lambda EY \cdot u_{EY}) = \lambda G \cdot F_G + \lambda Q \cdot F_Q + \lambda EX \cdot F_{EX} + \lambda EY \cdot F_{EY}$$

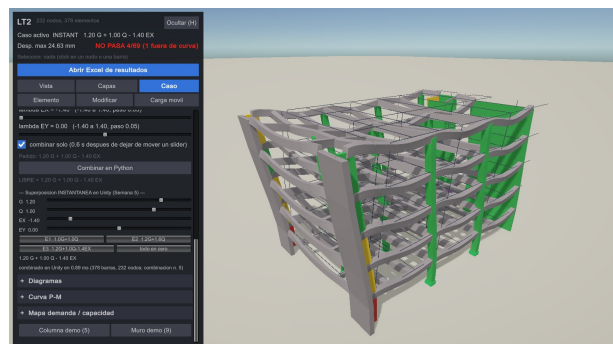
No hace falta volver a OpenSees para cada combinación: basta con escalar y sumar los cuatro casos que ya resolvió. Vale para desplazamientos, esfuerzos y reacciones. **No** vale para la demanda-capacidad, que se rehace (§4).

Qué hace Unity en cada movimiento del slider

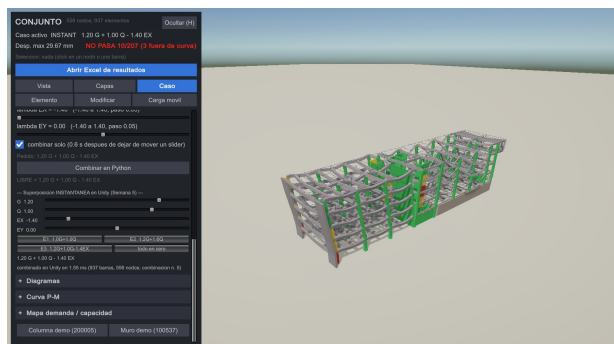
- **1. Escalar y sumar** los cuatro casos base con los λ de los sliders.
- **2. Retabular antes de sumar.** Los casos no vienen con las mismas estaciones: una viga cargada trae 9 puntos en G y Q, y 2 en EX y EY. Se interpola a la malla fina, y es **exacto**: sin carga repartida, el axial y el corte son constantes y el momento es una recta.
- **3. Rehacer la demanda.** Del vector de fuerzas combinado salen P y M con la misma regla de Python, y Mn se interpola en la curva P-M del anexo.

Los mismos tres estados, en los dos edificios

Estado	Factores	LT2	Conjunto
E1	1.0G + 1.0Q	8.79 mm · PASA todo (0/69)	25.04 mm · PASA todo (0/207)
E2	1.2G + 1.6Q	11.34 mm · 0/69	32.43 mm · 0/207
E3	1.2G + 1.0Q – 1.4EX	24.63 mm · NO PASA 4/69 (1 fuera de curva)	29.67 mm · NO PASA 10/207 (3 fuera de curva)
Tiempo del slider	medido en la app	0.9 a 4.4 ms (378 barras)	1.5 a 4.7 ms (937 barras)



LT2: sliders en E3. "Combinado en Unity en 0.89 ms (378 barras, 232 nodos)".



Conjunto: sliders en E3. "Combinado en Unity en 1.55 ms (937 barras, 558 nodos)".

Qué decir

“El modelo es lineal y elástico, así que la respuesta a una suma de cargas es la suma de las respuestas. Los cuatro casos base ya los resolvió OpenSees; el visor solo los escala y los suma, y rehace la demanda, que no es lineal. Con el edificio completo, 937 barras, tarda menos de 5 milisegundos.”

Dos caminos en la app, y por qué hay dos

	INSTANT (lo nuevo del LAB)	LIBRE (referencia)
Dónde combina	En Unity, en C#	En Python: POST /combinar
Respuesta	milisegundos, mientras se arrastra	0.6 s después de soltar + ida y vuelta
¿Necesita servidor?	No	Sí (la primera vez arma la base: 3.5 s LT2, 7–10 s conjunto)
Para qué sirve	La experiencia instantánea que pide el enunciado	Comprobar que INSTANT da lo mismo

Los botones **E1**, **E2** y **E3** de los sliders ponen los mismos factores que los estados precalculados por Python. E3 con los sliders y E3 precalculado dan la misma cabecera: **24.63 mm, NO PASA 4/69** en el LT2 y **29.67 mm, NO PASA 10/207** en el conjunto.

Cómo se verifica (y los dos errores que encontró)

`verificar_instantanea.py --registro` hace dos pruebas. Repite el algoritmo del visor en Python, con aritmética de 32 bits, en 10 juegos de λ . Y cruza contra Python **lo que la app real escribió** al mover los sliders: desplazamientos, fuerzas, P, M, Mn, el extremo que manda y PASA/NO PASA. **Pasa en los dos edificios** (registros en [semana05/capturas/](#) y [semana05_lab/capturas_conjunto/](#)).

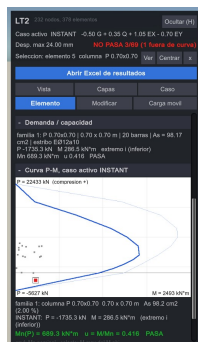
Error 1: la primera versión interpolaba con la x de cada estación, que viene redondeada a 4 decimales. Por la pendiente del momento daba $1.1\text{e-}2$ kN·m. Se interpola por fracción de índice: $1.6\text{e-}4$, bajo la cota $2.5\text{e-}4$.

Error 2: los sliders se **congelaban tras el primer movimiento** (el caso se registraba como `"combinacion"` y el hook solo reemplaza `"superposicion"`). La réplica en Python no podía verlo; el registro de la app sí. Hoy la versión sube 1 en cada movimiento.

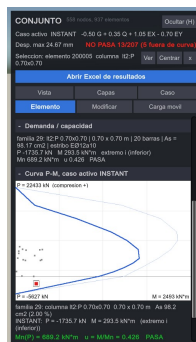
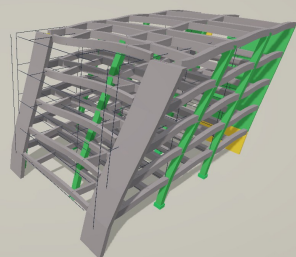
4. Demanda-capacidad dinámica (2 pts)

La demanda-capacidad **no es lineal**: $u = M/M_n$ y M_n depende de P a través de la curva. Por eso no se suma, se **rehace** en cada combinación:

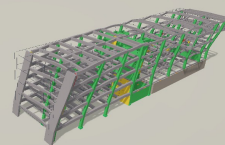
- De las fuerzas combinadas de cada barra salen **P** y **M** en sus dos extremos, y se queda el de mayor momento.
- $M_n(P)$** se interpola en la curva P-M de la sección, que Python calculó una vez con fibras ([comun/capacidad.py](#)). Ahora corre la sección **y su espejo** y se queda con el menor M_n : los muros del LT2 tienen el fierro descentrado y hay dos capacidades, una por sentido.
- $u = M / M_n$** . Si $u \leq 1$, PASA. El mapa D/C pinta cada barra según u y la cabecera cuenta.



LT2, λG negativo ($-0.50G + 0.35Q + 1.05EX - 0.70EY$): la columna 5 en tracción, $P = -1\,735\text{ kN}$, $M_n = 689\text{ kN}\cdot\text{m}$, $u = 0.416$.



Conjunto, los mismos λ : columna 200005, $P = -1\,736\text{ kN}$, $M_n = 689\text{ kN}\cdot\text{m}$, $u = 0.426$.



Qué decir

“La demanda-capacidad no es lineal, así que no se suma: se rehace. Del vector de fuerzas ya combinado salen P y M , y M_n se interpola en la curva P-M que calculó Python con fibras. Unity solo hace la aritmética, y está comprobada contra Python.”

Qué no pasa, en cada edificio (y por qué)

	LT2	Conjunto
NO PASA en E3	4 de 69: muros 14, 15, 16 y 29 (el 16 fuera de curva)	10 de 207: los mismos 4 muros del LT2 (2000xx) + 6 muros del antiguo (100397, 100411–413, 100508–509)
Fallan en alguna de las 15	9 elementos: todos muros . Las 40 columnas pasan en los 15 casos.	32 elementos: 30 muros y 2 columnas del antiguo (100042, 100043), solo con sismo en Y, $u \leq 1.20$
La combinación con más	0.9G – 1.4EX: 6/69	0.9G – 1.4EX: 16/207

- Por qué fallan los muros:** su armadura está incompleta en los planos o viene en texto que el extractor no lee (el atributo **CANT** vale 1 en 660 de 780 bloques; los M 0.60x2.92 van sin fierro). Su M_n sale con menos fierro del que probablemente tienen. El NO PASA marca **dónde falta información**, no un veredicto sobre el edificio.
- $u = 9999$** quiere decir que P quedó fuera del rango de la curva (tracción mayor que la que resiste el fierro modelado).
- Por qué 0.9G – 1.4E:** es la combinación que descarga el peso y da vuelta el axial sísmico. Manda en tracción de muros y columnas de borde; antes no estaba en la tabla (ahora son las 11 de NCh3171).

Cómo viaja un reanálisis

La pestaña **Modificar** edita el modelo en memoria ([EditorEstructura.cs](#)). **Recalcular** manda el modelo completo a [POST /analizar](#). El servidor arma K con OpenSees, resuelve G, Q, EX y EY, y devuelve desplazamientos, fuerzas, reacciones y la tabla de equilibrio. Unity **muestra la tabla que calculó el servidor**; no suma reacciones por su cuenta.

Qué decir

“Cambió K: los resultados guardados son de otra estructura, así que hay que reanalizar. En el conjunto da lo mismo que en el LT2 solo porque la junta es libre. Y ojo con la M3: la carga aplicada no cambió aunque la viga sea la mitad, porque el peso propio viene sumado en las cargas del modelo.”

Límites declarados del reanálisis desde la app

- **El peso propio no sigue a la sección:** las cargas de G vienen sumadas en el modelo.
- **Las curvas P-M no se rehacen:** si se cambia una sección con fierro, hay que reexportar el anexo. En la M3 no aplica: la viga 337 no tiene fierro extraído.
- **Material y área tributaria no tienen control en la app:** se cambian editando el modelo y reexportando.

6. Defensa: cuándo hay que reanalizar (2 pts)

Todo sale de $K \cdot u = F$: K es la rigidez (geometría, secciones, material, apoyos) y F las cargas. Vale igual para los dos edificios.

Las tres reglas

- **Si K cambia**, lo guardado es de otra estructura: hay que **reanalizar**.
- **Si K no cambia**, se escalan y suman casos resueltos: es superposición.
- **Si K no cambia pero F sí**, la pregunta es **"¿el F nuevo es combinación de casos ya resueltos?"**. Reanalizar es resolver un caso base nuevo, no volver a correr porque algo cambió.

Lo que cambia	¿K?	¿F?	¿Reanálisis?	Por qué
λ de un caso base (sliders)	no	no	No	Es superposición.
Todo un caso ($Q \times 1.5$)	no	sí	No	Es $\lambda Q = 1.5$.
Coefficiente sísmico C_s	no	sí	No	F es proporcional a C_s : $\lambda EX = C_s'/C_s$.
q o patrón en altura	no	sí	Sí	Cambia la forma de F piso a piso.
Carga de una viga	no	sí	Sí	No hay caso base resuelto con carga solo ahí.
Área tributaria	no	sí	Sí	Cambia la losa que baja a esa viga.
Borrar un elemento	sí	sí	Sí	Otra estructura, y se van sus cargas.
Apoyo	sí	no	Sí	Otra estructura.
Sección	sí	sí*	Sí , y su curva P-M	*Su peso (la app no lo actualiza). La capacidad no sale de $K \cdot u = F$.
Material uniforme	αK	no	No para esfuerzos	$u \rightarrow u/\alpha$ exacto; esfuerzos y reacciones iguales.
Material de una sección	sí	no	Sí	K cambia de forma no uniforme.

Lo que cambia	¿K?	¿F?	¿Reanálisis?	Por qué
El hormigón de un cuerpo del conjunto	sí	no	Sí , ese cuerpo	Con junta libre, solo cambia ese cuerpo; el otro queda igual.

Los números que respaldan la tabla

Prueba	LT2	Conjunto
Cs 0.10 → 0.20: corte basal	3 792.28 → 7 584.56 kN (×2.0000)	9 700.43 → 19 400.86 kN (×2.0000)
$\max EX(0.20) - 2 \cdot EX(0.10) $	1e-8 m y 6e-4 kN: un paso de redondeo	1e-8 m y 4e-4 kN: un paso de redondeo
G y Q con otro Cs	idénticos (0.0)	idénticos (0.0)
NO PASA en EX con Cs = 0.20	3 → 9 de 69	10 → 23 de 207

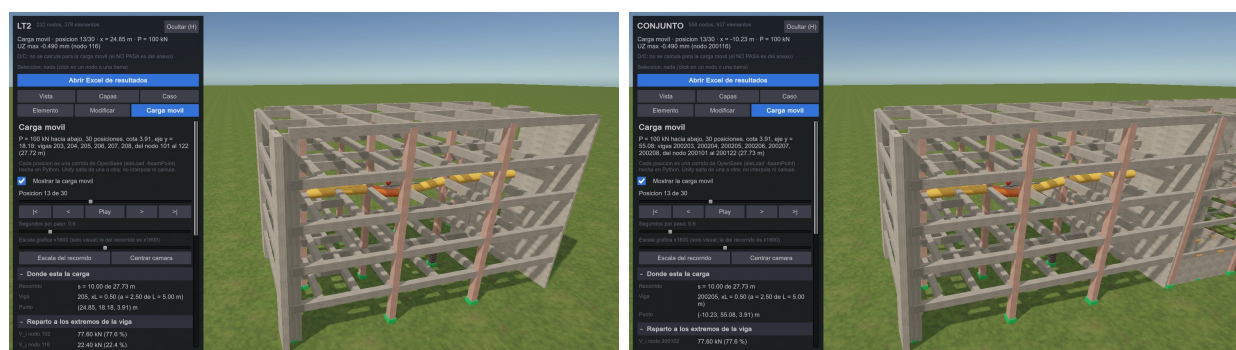
- **q y el patrón sí cambian la forma.** Con el patrón NCh433 las fracciones de V por piso pasan de 0.075, 0.139, 0.208, 0.276, 0.301 a 0.121, 0.126, 0.150, 0.194, 0.409. Ningún λ sobre EX produce eso.
- **Material uniforme.** f'_c de 35 a 140 MPa ($E \times 2$): $|u_2 - u/2| = 5e-9$ m, $\Delta f = 0.0$ kN.

Qué decir

“Subí el coeficiente sísmico de 0.10 a 0.20: ¿necesita reanálisis? No. Las fuerzas son proporcionales a Cs, así que EX(0.20) es exactamente 2·EX(0.10): es el slider en $\lambda EX = 2$. Lo rehicimos en Python en los dos edificios y dio lo mismo a un paso de redondeo, y eso prueba que el modelo es lineal. Lo que sí sería otro caso base es cambiar q o el patrón, porque cambian la forma de las fuerzas.”

7. Sidequest SQ4: carga móvil (recomendado)

Está a medias, y hay que decirlo así. La carga móvil existe en los dos edificios (pestaña **Carga móvil**): 30 posiciones resueltas en OpenSees sobre el eje de vigas 203–208 del LT2 (200203–200208 en el conjunto), con el reparto y la conservación verificados. Pero **no sigue al usuario**: la posición se elige con un control. Falta tomar la posición del usuario, encontrar el paño de losa que la contiene y resaltar sus vigas receptoras; los datos para hacerlo están (`areas_tributarias`).



LT2: posición 12, sobre la viga 205.

Conjunto: posición 12, viga 200205.

8. Preguntas probables y respuestas cortas

El profesor puede elegir a cualquiera de los tres para explicar cualquier parte.

Pregunta	Respuesta
¿Unity calcula la estructura?	No. OpenSees resuelve en Python. Unity escala, suma e interpola resultados ya resueltos, y esa aritmética está verificada contra Python en los dos edificios.
¿Por qué se puede superponer?	Porque el modelo es lineal elástico: $K \cdot u = F$ con K constante. Dejaría de valer con material no lineal, $P-\Delta$ o contacto (por ejemplo, si la junta se cerrara y los cuerpos chocaran).
¿Por qué no se suma la D/C?	Porque M_n depende de P . Se suman las fuerzas y después se recalcula $u = M/M_n(P)$.
¿Cómo saben que el slider no miente?	E3 con los sliders y E3 precalculado por Python dan la misma cabecera (24.63 mm y 4/69 en el LT2; 29.67 mm y 10/207 en el conjunto). Y <code>verificar_instantanea.py --registro</code> cruza lo que la app escribió contra Python.
¿Cómo se unen los dos cuerpos?	<code>conjunto/armar.py</code> : calce (el LT2 se corre a 5 cm del antiguo), reenumeración (1xxxxx y 2xxxxx) y junta libre. Cada sección lleva su hormigón. G del conjunto = la suma exacta.
¿Por qué M1 da igual en el conjunto?	Porque la junta es libre: no hay elementos entre cuerpos, así que editar el LT2 no cambia el antiguo. Con junta cerrada (o un elemento de contacto) eso ya no valdría.
¿De dónde sale la curva P-M?	De <code>comun/capacidad.py</code> : sección de fibras de OpenSees, un análisis momento-curvatura por cada P , y el momento máximo. Se corre la sección y su espejo y se toma el menor.
¿Por qué los muros no pasan?	Su fierro está incompleto en los planos, así que su M_n sale bajo. En el LT2 las 40 columnas pasan en los 15 casos; en el conjunto fallan solo 2 columnas del antiguo, con sismo en Y.
¿Por qué 15 casos y no 9?	Faltaba el sismo en los dos sentidos y la combinación $0.9G \pm 1.4E$, que manda en tracción. Son las 11 combinaciones de NCh3171 más los 4 casos base.
¿Qué es el diafragma rígido?	Cada piso se mueve como cuerpo rígido en su plano (se traslada y gira). En OpenSees va con <code>constraints('Transformation')</code> . El conjunto tiene 10, 5 por cuerpo.
¿Por qué no se suman todas las reacciones?	Un nodo de diafragma también reacciona a su restricción, que es interna. Sumar la columna entera da el corte basal doble. <code>calcular.equilibrio()</code> separa por GDL.
¿Qué fuerzas internas usan?	<code>eleResponse(tag, 'localForce')</code> , en ejes locales. Nunca <code>eleForce</code> , que es global.
¿Qué es supuesto y qué sale del plano?	Se distingue en el dato (<code>origen, _supuesto</code>). Supuestos: el sismo (Cs, q, patrón), el Ø25 de los pilares del LT2, el Ø22 del pilar 70x70 del antiguo (el menor que cumple la cuantía mínima), la junta de 5 cm y los dinteles.

9. El día de la defensa

La noche anterior (una vez, con Unity cerrado)

```
.\.venv\Scripts\python.exe comun\verificar_todo.py # 47 de 47 EN OK; 2 a 8 min; deja el LT2
```

Antes de que llegue el profesor (5 minutos)

Paso	Si muestras el LT2	Si muestras el conjunto
1. Anexos	lanzar_unity.py preparar lt2	lanzar_unity.py preparar conjunto
2. Servidor (otra terminal, se deja abierta)	servidor_s5.py	servidor_s5.py --edificio conjunto
3. Esperar	[base] lt2 armada en ~4 s	[base] conjunto armada en ~10 s
4. App	lanzar_unity.py app lt2	lanzar_unity.py app conjunto

Todos con `.\.venv\Scripts\python.exe` delante y la ruta `comun\` o `semana05\`. **preparar**, no sincronizar: la suite deja los anexos del LT2, y **sincronizar** conjunto solo copia. En la terminal del servidor salen muchos `analyze failed`: son los avisos esperados de las curvas P-M.

El orden de la demo (10 minutos)

#	Bloque	Qué hacer
1	Interactividad	Navegar → seleccionar una viga → Capas → Caso 1.2G+1.6Q → deformada y diagramas → Columna demo → Mapa D/C.
2	Superposición	Caso → Superposición → bajar hasta <i>Superposición INSTANTÁNEA</i> . Mover EX y G. Luego E3 en los sliders y E3 precalculado: misma cabecera.
3	D/C dinámica	Columna demo + Curva P-M. EX de 0 a 1.4; G negativo → tracción.
4	Modificación	Al final , porque deja el anexo desactualizado: M3 y M1 en la pestaña Modificar (IDs de la tabla del §5). M2 y M4 por comando si preguntan.
5	Criterios	La tabla del §6 y cerrar con <code>verificar_instantanea.py --registro</code> .

Entrega en Canvas

El LAB está en el **PR #8** (rama `semana05-lab`), que todavía no está mergeado en `main`. Entregar el hash del último commit de la rama, o el del merge si Pedro lo mergea antes.

Enlaces: github.com/bitscochits/A1P3_Grupo_7/pull/8 y el README del LAB en ese commit.

Si algo sale mal en vivo

Síntoma	Qué hacer
No encuentro los sliders instantáneos	Están al final de <i>Superposición (Semana 5)</i> , debajo de <i>Factores libres</i> . Se llaman G, Q, EX, EY.
Muevo el slider y no pasa nada	Es el de LIBRE ($\lambda G = \dots$), que espera al servidor. Usar los de abajo.
"OJO: semana04.json no se copió" o "el anexo es de otro modelo"	<code>lanzar_unity.py</code> preparar y reabrir la app.
El servidor dice "WinError 6"	Era un servidor viejo: cerrarlo (Ctrl+C) y volver a lanzarlo. Está arreglado desde 484582e.
La cabecera dice "modelo editado"	Quedó una M1 o M3 de una prueba anterior: reabrir la app.
Falla la M3 desde la app	Por comando: <code>reanalisis_demo.py lt2 --seccion 337 "V 0.30x0.80" --nodo 186</code> (en el conjunto: <code>conjunto --seccion 200337 "lt2:V 0.30x0.80" --nodo 200186</code>).